

Project Report:

End-to-end data pipeline project

Team Members: Joshua, Jacob, Norman, Puay Hon, Lim Yong.

20th June, 2025

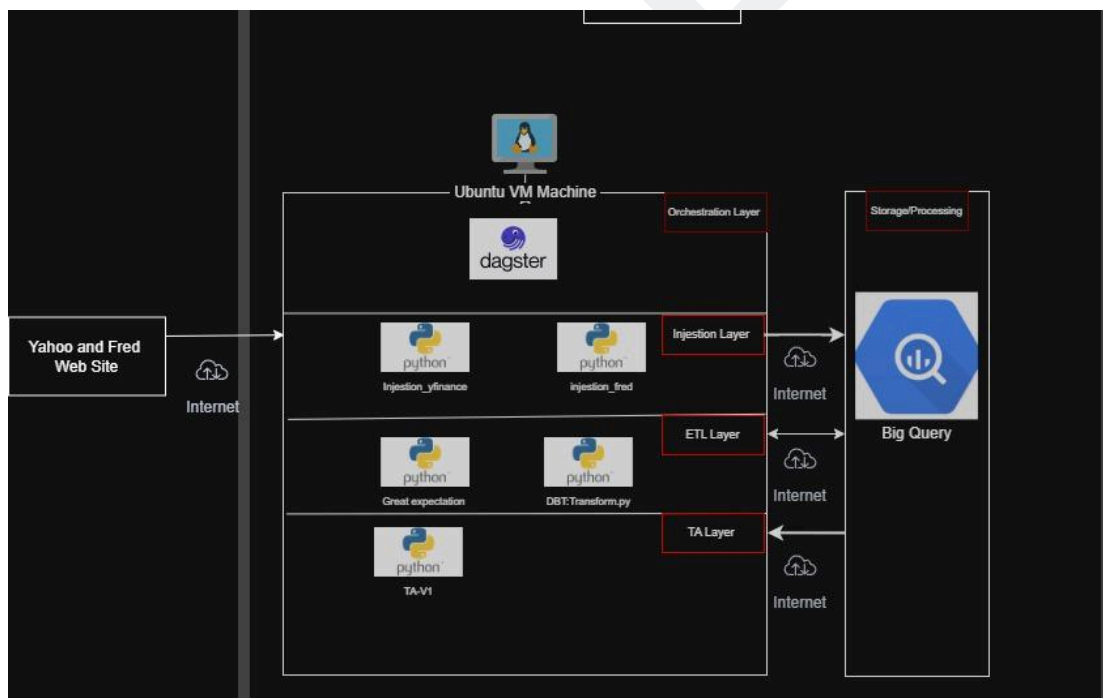


Data architecture.....	2
Data Ingestion.....	3
Data warehouse design.....	5
ETL Pipeline.....	6
Data Quality Testing.....	8
Data pipeline orchestration.....	10
Data Analysis with Python.....	13
User Story 1.....	13
User Story 2.....	19
User Story 3: Pattern-Driven Signal Detection Using Multi-Indicator Analysis.....	20
Challenges.....	24

Data architecture

Data architecture is the strategic blueprint for managing and structuring data across an organization, encompassing everything from data modeling and storage methods to integration processes, governance frameworks, and data flow strategies. A well-designed architecture enhances performance and efficiency, enabling quick data retrieval, seamless scalability, and robust security, much like keeping your workspace tidy and secure so you always find what you need swiftly.

Crucially, effective data architecture matters to IT as it empowers smarter, data-driven decisions, ensures compliance with regulatory requirements, and provides flexibility to integrate emerging technologies. It serves as a foundational enabler for innovation and agility, future-proofing IT systems to support ongoing growth and evolution, similar to constructing a resilient yet adaptable infrastructure that comfortably supports future upgrades and advancements.



This diagram depicts a data pipeline architecture hosted on Google Cloud Platform, using an Ubuntu VM machine. Dagster orchestrates data flows, managing data ingestion from Yahoo and Fred websites via Python scripts. The architecture features clearly defined layers: an ingestion layer (yfinance and FRED data), an ETL layer utilizing Great Expectations for data validation and DBT for transformations, and a technical analysis (TA) layer. Processed data is stored and managed in BigQuery, establishing a streamlined workflow for effective data analytics.

Data Ingestion

```
### section here is to download yfinance jacob's code this is usable. Includes Peihan's original code.
import yfinance as yf
import pandas as pd
from google.cloud import bigquery
import json
import os

####Note that the stock_config.json must be in the same directory is your py file

# Load stock config
with open("stock_config.json") as f:
    stock_config = json.load(f)

# Load BigQuery config
with open("bq_config.json") as f:
    bq_config = json.load(f)

##extract ticker values
ticker_symbols = stock_config["ticker"]
period = stock_config["period"]

# Extract BigQuery values
project_id = bq_config["project_id"]
dataset_id = bq_config["dataset_id"]
table_id = bq_config["table_id"]
os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = bq_config["key"] # service account private key

# Collect data for each ticker
all_data = []

for symbol in ticker_symbols:
    df = yf.Ticker(symbol).history(period=period)
    df["ticker_symbol"] = symbol # Add a column to identify the ticker
    all_data.append(df)
```

The extraction method involves fetching financial data from online sources like Yahoo Finance using Python scripts. The data extraction process includes

Configuration and Initialization:

The script loads configurations from JSON files, specifying ticker symbols, periods, and BigQuery credentials.

Data Retrieval:

Using the yfinance library, financial data such as stock prices and technical indicators are downloaded for specified tickers.

```
# Calculate indicators
df['rsi_14'] = ta.momentum.RSIIndicator(close=close_prices, window=14).rsi()
df['sma_50'] = ta.trend.SMAIndicator(close=close_prices, window=50).sma_indicator()
df['ema_50'] = ta.trend.EMAIndicator(close=close_prices, window=50).ema_indicator()
```

Data Processing:

The retrieved data transforms, such as calculating technical indicators (RSI, SMA, EMA), formatting, and cleaning.

```
# 3. Initialize BigQuery client
client = bigquery.Client(project=project_id)

# 4. Load data into BigQuery
job_config = bigquery.LoadJobConfig(
    ## section here is set to over ride the data. Using append complicates the code
    write_disposition="WRITE_TRUNCATE", # or WRITE_TRUNCATE or # "Write_Append"
    autodetect=True,
    source_format=bigquery.SourceFormat.PARQUET
)

# Load DataFrame to BigQuery
job = client.load_table_from_dataframe(
    combined_df, f"{project_id}.{dataset_id}.{table_id}", job_config=job_config
)
job.result() # Wait for completion # or use .history(start=..., end=...
```

Data Loading:

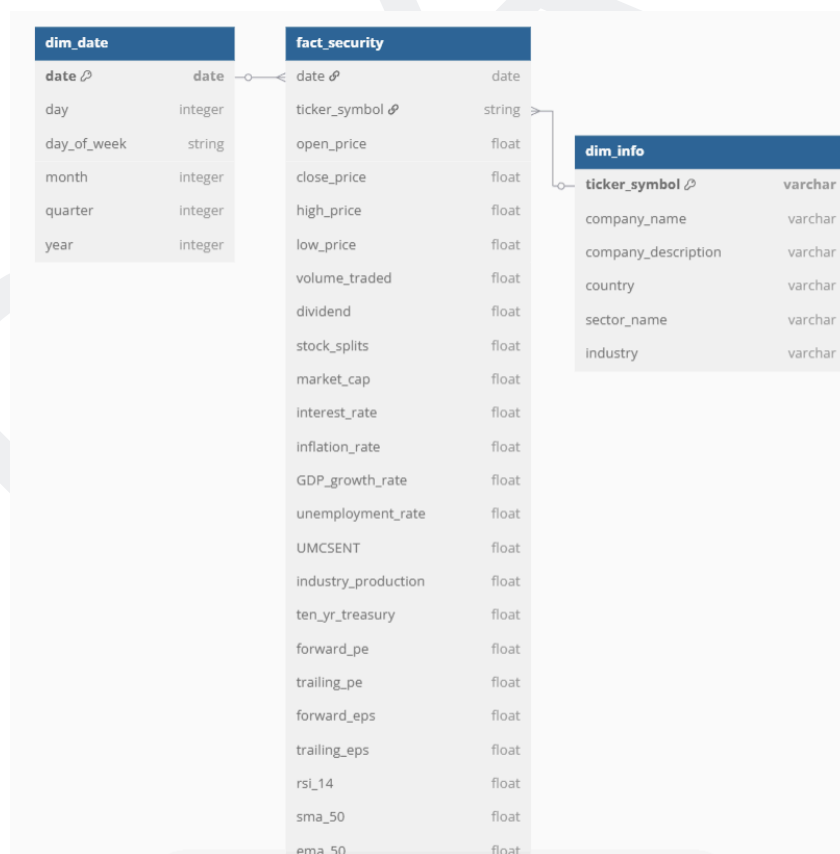
Processed data is then uploaded to a BigQuery dataset.

Data warehouse design

A data warehouse, such as Google's BigQuery, is a centralized repository designed to store large volumes of structured and semi-structured data from diverse sources. BigQuery is cloud-based, offering powerful tools for efficient storage, quick querying, and analytics on massive datasets. Its serverless architecture allows easy scaling, eliminates the need for complex infrastructure management, and provides high-speed SQL queries and seamless integration with other cloud services and analytical tools.

Benefits of BigQuery:

- **Scalability:** Easily handles large data volumes without manual intervention.
- **Performance:** Fast SQL queries through distributed, columnar storage.
- **Cost Efficiency:** Pay-per-query and storage models reduce operational costs.
- **Real-Time Analytics:** Supports streaming data for immediate insights.
- **Security and Compliance:** Offers robust access control and encryption.

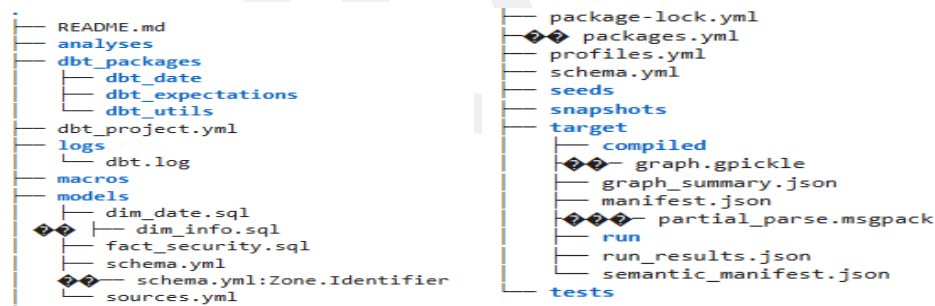


The star schema is a popular database schema optimized for analytical queries, consisting of a central fact table surrounded by multiple dimension tables. In your provided schema, fact_security is the fact table, storing measurable data (like stock prices and economic indicators). It's linked to dimension tables (dim_date, dim_info) that provide descriptive attributes (date attributes, company information). This schema structure ensures the data warehouse efficiently supports sophisticated analytics, strategic reporting, and informed decision-making.

Why Star Schema is Used:

- **Simplified Queries:** Queries become simpler and more efficient, significantly reducing join complexities.
- **Enhanced Performance:** Facilitates faster query execution and improved analytical response times.
- **Intuitive Data Structure:** Easier to understand, maintain, and use for business intelligence and reporting.
- **Optimized for OLAP (Online Analytical Processing):** Specifically designed for quick data slicing and dicing.

ETL Pipeline



To ensure future flexibility in data ingestion, DBT models are designed to support various data sources, including CSV files and other offline formats. This elasticity allows teams to adapt quickly as new data pipelines or formats are introduced without overhauling the entire transformation logic.

DBT pairs seamlessly with packages like **dbt-expectations**, providing a powerful layer for data testing. By integrating these tests into the transformation process, teams can validate data quality in real time, catching issues like null values, duplicates, or broken relationships before they impact downstream analytics.



Compared to native scripting languages tied to specific tools, SQL remains a universal and powerful language for data transformation. DBT takes this further by offering a **structured framework** that standardizes transformations, making it easier to manage even the most complex workflows.

One of DBT's key strengths lies in its **reusability and modularity**. Models can reference one another in a layered approach, reducing redundancy and improving maintainability across projects. As business logic evolves, changes in one model can propagate consistently throughout the entire pipeline.

Finally, DBT supports **built-in testing capabilities** such as unique, not null, and relationship tests. These checks ensure that data quality is enforced at every stage of development and automatically run during deployments. The availability of **open-source packages** like `dbt_utils` and `dbt_expectations` further simplifies repetitive tasks—such as date transformations, assertions, and format validation—enhancing developer productivity and confidence.

Data Quality Testing

DBT Testing Script Summary

1. fact_security (Fact Table)
 - a. Primary Key: fact_security_id (unique, not null)
 - b. Core Tests:
 - i. Date validation (not null, $\leq$ current date, links to dim_date)
 - ii. Ticker symbol validation (not null, links to dim_info)
 - iii. Price fields validation (open/close/high/low prices not null)
 - iv. Volume validation (not null, float type)
2. Financial Metrics:
 - a. Validates data types for dividends, splits, EPS, P/E ratios
 - b. Technical indicators (EMA/SMA/RSI) as float types
3. Macroeconomic Fields:
 - a. Validates float types for GDP, sentiment, inflation, rates
4. dim_info (Dimension Table)
 - a. Primary Key: ticker_symbol (unique, not null, string)
 - b. Company Attributes:
 - i. Validates not null for name, description, country
 - ii. Ensures uniqueness for name and description
 - iii. Validates string types for all text fields
5. Classification:
 - a. Sector and industry fields (not null, string)
6. dim_date (Dimension Table)
 - a. Primary Key: date (unique, not null, date type)
 - b. Validation:
 - i. Dates must be $\leq$ current date
 - ii. Proper date format enforcement

Key Testing Patterns:

- All primary keys: unique + not_null
- Critical business fields: not_null constraint
- Type validation for all numeric/date/string fields
- Referential integrity via relationships tests
- Temporal validity checks (date ranges)

Coverage Highlights:

- 100% primary key validation
- Core financial metrics quality checks

- 
- Dimensional referential integrity
 - Data type consistency enforcement
 - Business rule validation (e.g., valid date ranges)

DRAFT

Data pipeline orchestration

Dagster is an open-source data orchestrator platform designed to manage, schedule, and monitor complex data pipelines. It ensures data reliability, reproducibility, and ease of management by offering a clear and structured approach to pipeline orchestration.

Uses and Benefits of Dagster:

- **Pipeline Management:** Simplifies creation, execution, and monitoring of data pipelines.
- **Job Scheduling:** Uses cron-based scheduling to automate pipeline execution at specified intervals.
- **Data Reliability:** Provides clear logging, error handling, and retry mechanisms.
- **Ease of Use:** Offers an intuitive Pythonic API, making it accessible to developers and data engineers.
- **Scalability:** Manages both simple and complex data workflows effortlessly.

How Data is Orchestrated Using Dagster:

In the provided setup:

```
@job
def upload_yfinance_job():
    run_yfinance_script()
    run_stock_info_script()
    run_fred_script()

@job
def dbt_job():
    run_dbt_yfinance()

@job
def ta_compute():
    run_ta_script()

@job
def dbt_test():
    run_dbt_test()
```

1. Jobs Definition:

Dagster organizes tasks into defined jobs (`upload_yfinance_job`, `dbt_job`, `dbt_test`, `ta_compute`). These jobs wrap Python scripts or commands using Dagster's decorators (`@job`, `@op`).

```
@op
def run_yfinance_script():
    try:
        subprocess.run(
            ["python", "/home/biscuit/NTU-Project-Data-Science-AI/ingestion_pipe/ingestion_yfinance.py"],
            check=True
        )
    except subprocess.CalledProcessError as e:
        print(f"Error running ingestion_yfinance.py: {e}")
        raise

@op
def run_stock_info_script():
    try:
        subprocess.run(
            ["python", "/home/biscuit/NTU-Project-Data-Science-AI/ingestion_pipe/ingestion_stock_info.py"],
            check=True
        )
    except subprocess.CalledProcessError as e:
        print(f"Error running ingestion_stock_info.py: {e}")
        raise
```

2. Operations (ops):

Each task within the job, such as executing scripts (`yfinance`, `fred`, `stock_info`) and DBT commands (`run_dbt_yfinance`, `run_dbt_test`), is encapsulated within an `@op` function, ensuring clear modularity and maintainability.

```

# Run every day at 7:30 AM
daily_schedule = ScheduleDefinition(
    job=upload_yfinance_job,
    cron_schedule="30 7 * * *",
    name="daily_upload_yfinance_job",
    execution_timezone="Asia/Singapore"
)

daily_ta_schedule = ScheduleDefinition(
    job=ta_compute,
    cron_schedule="37 7 * * *",
    name="daily_compute_ta",
    execution_timezone="Asia/Singapore"
)

daily_dbt_schedule = ScheduleDefinition(
    job=dbt_job,
    cron_schedule="45 7 * * *",
    name="daily_dbt_job",
    execution_timezone="Asia/Singapore"
)

daily_dbt_test_schedule = ScheduleDefinition(
    job=dbt_test,
    cron_schedule="0 8 * * *",
    name="daily_dbt_test",
    execution_timezone="Asia/Singapore"
)

```

3. Scheduling:

Jobs are scheduled to run automatically using cron schedules. For instance:

- **upload_yfinance_job** runs daily at 7:30 AM (SGT).
- **ta_compute** executes daily at 7:37 AM.
- **dbt_job** is set for daily execution at 7:45 AM.
- **dbt_test** executes daily at 8:00 AM.

4. Execution & Monitoring:

Dagster manages execution, handles errors gracefully, and provides logging to easily track and debug job runs.

Data Analysis with Python

Disclaimer

This project is for educational and entertainment purposes only.

The analyses, charts, and commentary provided herein are part of an academic data science project. They are ****not intended as financial, investment, or trading advice****.

Although real financial data is used, the information and models presented ****do not guarantee accuracy, completeness, or future performance****.

The author(s) of this project are ****not licensed financial advisors****. Any investment-related observations are purely hypothetical and for illustrative purposes.

Please do not use this content as a basis for investment decisions. **** Always seek advice from a qualified financial professional before making any financial decisions**

User Story 1



Sector Rotation Analysis Using ETF Data (2020–2025)

1. 🎯 Objective of the Analysis

- Visualize sector rotation across economic cycles.
- Track sector performance using sector ETFs.
- Identify patterns in sector leadership around key macro events (e.g., Fed decisions, inflation).
- Support portfolio decisions with data-driven insights.

2. 📖 What Is Sector Rotation?

Sector rotation refers to the cyclical movement of capital among sectors during different economic phases. Investors shift between sectors based on:

- Growth expectations
- Interest rates

- Inflation trends

Textbook rotation sequence:

<u>Cycle Phase:</u>	<u>Leading Sectors</u>
Early Recovery:	Technology, Consumer Discretionary, Communication Services
Mid Expansion:	Industrials, Materials, Energy
Late Expansion:	Consumer Staples, Health Care, Utilities
Recession:	Financials, Real Estate

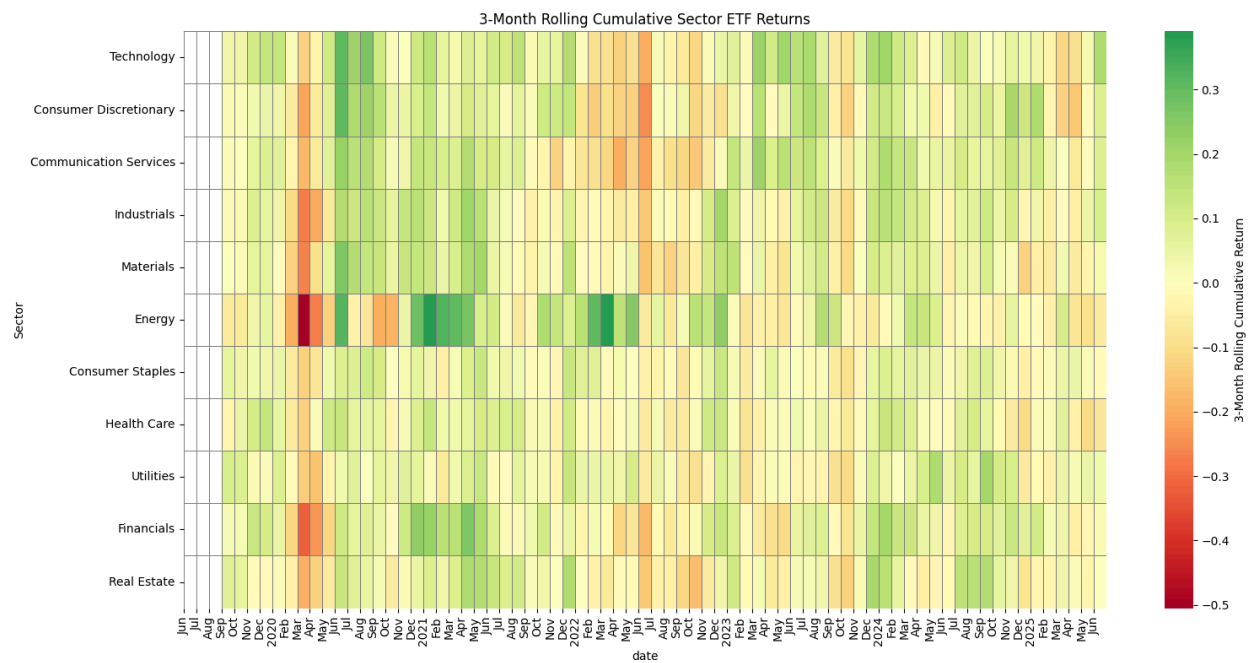
3. 🛠️ Tools & Data

- ETFs: Sector proxies like XLK (Technology), XLF (Financials), etc.
- Data Source: Yahoo Finance via yfinance Python package.
- Timeframe: 2019-06-01 to 2025-06-01
- Data Handling: pandas, matplotlib, seaborn
- Data Source (alternative): Retrieve ETF tickers dynamically from BigQuery.

4. 📊 Methodology

a. Monthly Returns & 3-Month Rolling Cumulative Returns

- Resample ETF closing prices to monthly.
- Compute percentage change for monthly returns.
- Apply a 3-month rolling product for cumulative trends.

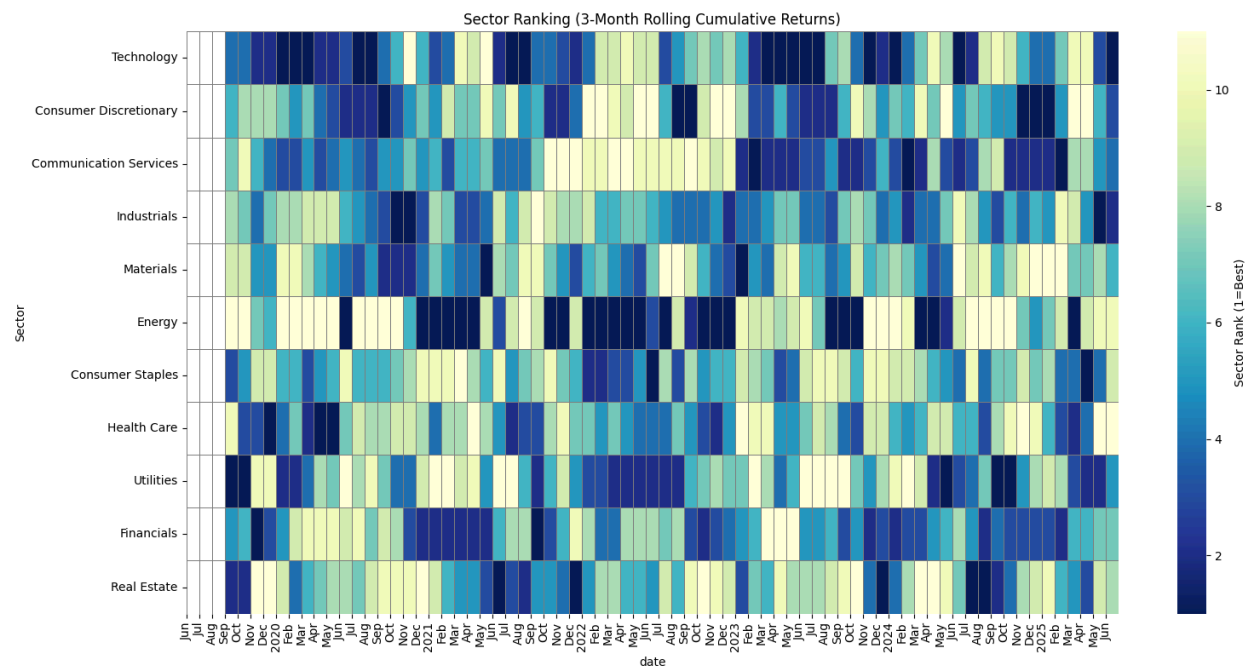


b. Rolling Heatmap of Returns

- seaborn.heatmap to display rolling 3-month returns.
- Color-coded to identify hot and cold sectors over time.

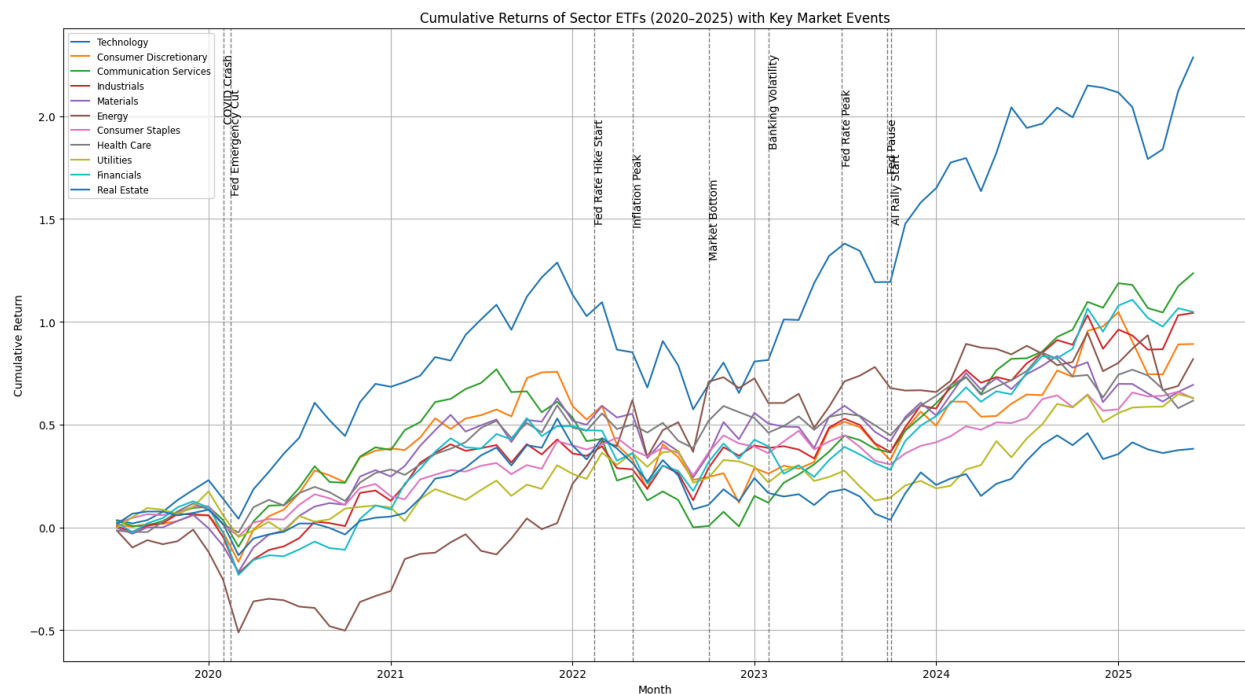
c. Sector Ranking Heatmap

- Rank sectors each month by performance (1 = Best).
- Reveals sector leadership rotation visually.



d. Cumulative Return Line Plot

- Tracks sector performance over 5 years.
- Annotated with macro events like:
 - COVID Crash
 - Fed Rate Hikes
 - Inflation Peak
 - AI Rally Start



5. 🧠 Key Insights

Event Observation

COVID Crash (Mar 2020):	All sectors dropped; Tech rebounded fastest
Fed Rate Hike Cycle (Mar 2022):	Defensive sectors (Staples, Health Care) outperformed
Inflation Peak (Jun 2022):	Market bottom followed in Oct 2022
AI Rally (Nov 2023):	Tech, Comm Services resurged strongly
Utilities & Staples:	Provided downside protection during volatility

- The heatmaps and plots show clear sector rotation aligned with macro phases.
- Defensive sectors shine in uncertainty; cyclicals lead in growth phases.

6. 🧠 Real-World Applications

- 🧭 Portfolio Strategy: Align with top-performing sectors per cycle.
- 🛡️ Risk Management: Avoid sectors hurt by rate hikes or inflation.

- 
-  17 Macro Timing: Anticipate rotation using economic indicators (e.g., CPI, FOMC).

Summary

- Sector rotation analysis helps investors navigate market cycles.
- Using rolling returns, ranks, and event annotations enhances interpretability.
- This framework can scale with additional macroeconomic and valuation layers.

Developed using Python (yfinance, pandas, seaborn, matplotlib) and optionally powered by BigQuery.

User Story 2

Dafds

DRAFT

User Story 3: Pattern-Driven Signal Detection Using Multi-Indicator Analysis

Traders often struggle to navigate high market volatility driven by economic data releases, Federal Reserve decisions, geopolitical tensions, and inflation reports, increasing the risk of buying at peaks and selling at lows. There is a need for a strategy that enables timely entry and exit decisions to reduce risk and maximize returns during turbulent market conditions.

Objectives of the Stock Pattern Analysis Project

1. **To analyze stock price patterns** using technical indicators such as moving averages, Bollinger Bands, and RSI to identify trends, support/resistance levels, and buy/sell signals.
2. **To examine the impact of macroeconomic indicators** by exploring correlations between stock prices and variables like GDP growth, inflation, interest rates, and unemployment.
3. **To develop an interactive dashboard** that allows users to filter, compare, and visualize stock data for deeper analysis and interpretation.

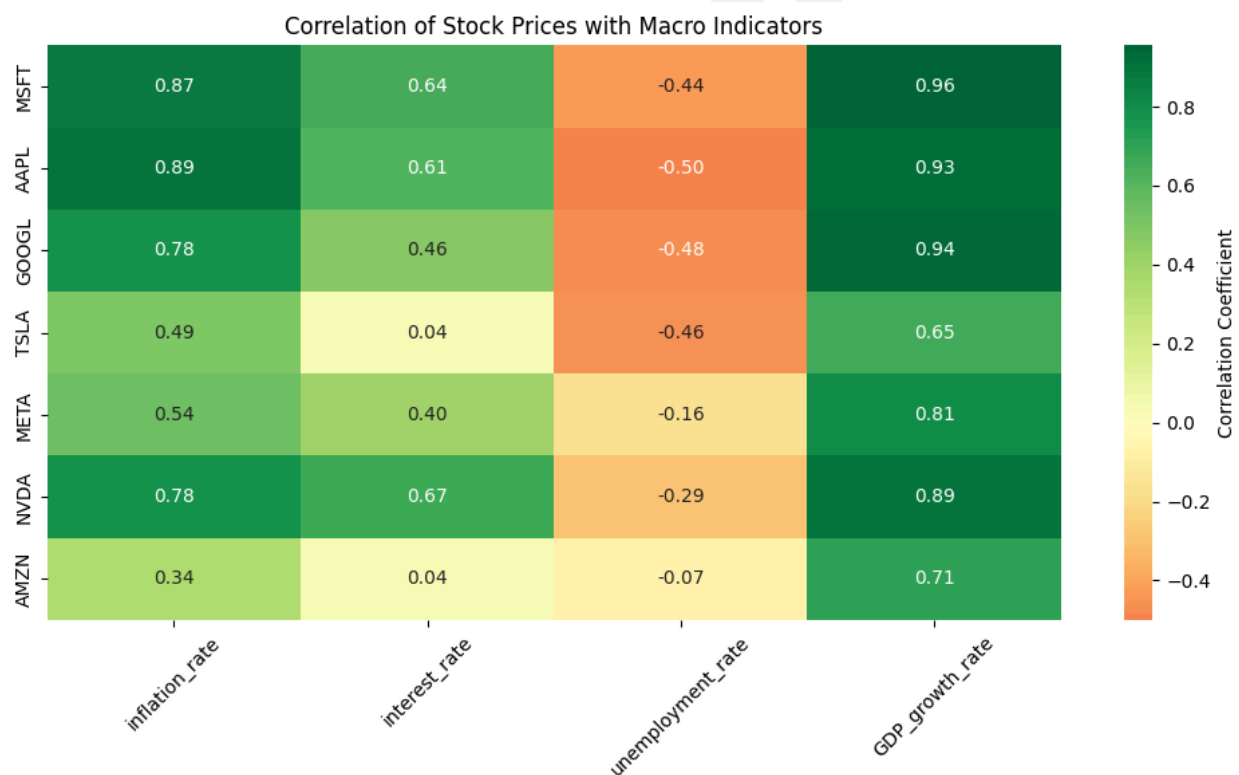
	date	ticker_symbol	close_price	GDP_growth_rate	inflation_rate	interest_rate	unemployment_rate
0	2019-06-19	MSFT	128.422348	NaN	NaN	NaN	NaN
1	2019-06-19	AAPL	47.518715	NaN	NaN	NaN	NaN
2	2019-06-19	GOOGL	54.896267	NaN	NaN	NaN	NaN
3	2019-06-19	TSLA	15.095333	NaN	NaN	NaN	NaN
4	2019-06-19	META	186.457092	NaN	NaN	NaN	NaN
...	...	...	...	...	...	...	...
10551	2025-06-17	NVDA	144.119995	NaN	NaN	NaN	NaN
10552	2025-06-17	META	697.229980	NaN	NaN	NaN	NaN
10553	2025-06-17	AMZN	214.820007	NaN	NaN	NaN	NaN
10554	2025-06-17	AAPL	195.639999	NaN	NaN	NaN	NaN
10555	2025-06-17	MSFT	478.040009	NaN	NaN	NaN	NaN

10556 rows x 7 columns

The dataset containing historical information on the "Magnificent 7" stocks—namely AAPL, GOOGL, AMZN, MSFT, TSLA, META, and NVDA—was extracted from Google BigQuery. A Python script was used to execute the query via the BigQuery client library, utilizing a secure JSON credentials file for authentication. Configuration parameters such as the project ID, dataset ID, and table ID were dynamically loaded from external JSON files to ensure modularity and reusability. The SQL query was structured to retrieve daily stock price data along with key economic indicators, including GDP growth rate, inflation rate, interest rate, and unemployment rate, for the period between June 19, 2019, and June 18, 2025. The query results were then converted into a Pandas DataFrame for subsequent analysis and visualization.

Correlation heatmap

The correlation analysis between stock closing prices and macroeconomic indicators—namely inflation rate (CPI), interest rate (federal funds rate), unemployment rate, and GDP growth—reveals meaningful relationships. Notably, AAPL, MSFT, and GOOGL show strong positive correlations with CPI and GDP growth, suggesting that these tech giants tend to perform well during periods of economic expansion and rising consumer prices. On the other hand, the unemployment rate exhibits a consistently negative correlation across most stocks, indicating that higher unemployment levels generally dampen stock performance. Interest rates show moderate to weak correlation, with NVDA showing the strongest sensitivity. These patterns highlight how macroeconomic shifts can serve as influential drivers of equity performance, particularly for growth-oriented technology firms.



Technical Analysis

To generate reliable buy and sell signals, a combination of technical indicators and normalized macroeconomic data was employed. Moving Averages (e.g., MA20, MA50) were used to smooth out price fluctuations and identify underlying trends. MACD (Moving Average Convergence Divergence) helped detect momentum shifts by measuring the difference between short-term and long-term EMAs. RSI (Relative Strength Index) was used to identify overbought or oversold conditions, typically signaling potential reversals. Bollinger Bands captured price volatility, with breakouts beyond the upper or lower bands often indicating entry or exit opportunities. In parallel, economic indicators such as inflation, interest rates, unemployment, and GDP were normalized using Min-Max scaling to create a composite confidence score, reflecting overall economic sentiment. A momentum surge, indicated by the convergence of bullish technical signals and high confidence scores, was used to trigger buy signals, while weakening momentum or negative macroeconomic trends guided sell decisions.



The visualizations present a comprehensive technical analysis of AAPL stock, highlighting both long-term trends and short-term entry signals. Key indicators such as Exponential Moving Average (EMA20), Bollinger Bands, RSI, and MACD were layered onto the price chart to detect momentum shifts and trading opportunities. The buy signals (green triangles) and sell signals (red triangles) were generated based on a convergence of multiple conditions: MACD crossovers, RSI thresholds, price position relative to Bollinger Bands, and momentum bursts.



The zoomed-in view (July to October 2023) reveals precise signal timing and price movements, showcasing how early entries can be detected when technical indicators align. This multi-indicator framework strengthens the confidence in signal validity, reducing false positives and supporting more informed trading decisions.

Challenges

Where	Issue	Why	Solution
Dagster Sensor	Sensor failed	Mismatch of package version, package compatibility	
The date column from stock price includes a timestamp.	BigQuery regards date with time information included as a timestamp. This cause issue with other table that only uses Date datatype	Default behaviour	Make use of the DBT Dim table to transform the timestamp to the date datatype.
Dagster Deployment via @assets	difficult to debug and time consuming	code has to be refactored and tested. bugs cannot be easily fixed.	Use a Dagster wrapper and @ops. Code doesn't need to be refactored. Can be directly called from a Py or jupyter notebook. Easier for step-by-step debugging.